

Convexity and KKT Conditions in Python

Week 2 Seminar

A. D. Ignatov
HSE University, Faculty of Computer Science

2026–2027

Seminar plan

1. SymPy basics and symbolic derivatives.
2. Convexity tests for sets and functions.
3. Smoothness and strong convexity constants for quadratics.
4. Equality constraints and Lagrange multipliers.
5. Inequality constraints and KKT conditions.
6. SciPy verification and dual sensitivity.

All tasks appear in the accompanying notebook with executable scaffolding.

Lecture recap: convexity

For a convex set C ,

$$x, y \in C, t \in [0, 1] \implies (1 - t)x + ty \in C.$$

For differentiable $f : C \rightarrow \mathbb{R}$,

$$f \text{ convex} \iff f(y) \geq f(x) + \nabla f(x)^\top (y - x).$$

If $f \in C^2$ on an open convex set,

$$f \text{ convex} \iff \nabla^2 f(x) \succeq 0.$$

Lecture recap: curvature constants

For a twice differentiable function on a convex domain:

$$\mu I \preceq \nabla^2 f(x) \preceq LI$$

implies μ -strong convexity and L -smoothness.

For $f(x) = \frac{1}{2}x^\top Qx + b^\top x + c$ with symmetric Q ,

$$\mu = \lambda_{\min}(Q), \quad L = \lambda_{\max}(Q).$$

The ratio $\kappa = L/\mu$ measures conditioning when $\mu > 0$.

Lecture recap: KKT

For $\min f(x)$ subject to $g_i(x) \leq 0$ and $h_j(x) = 0$:

$\nabla_x \mathcal{L}(x, \lambda, \nu) = 0$	stationarity,
$g_i(x) \leq 0, h_j(x) = 0$	primal feasibility,
$\lambda_i \geq 0$	dual feasibility,
$\lambda_i g_i(x) = 0$	complementary slackness.

In a convex problem, any KKT point is globally optimal.

SymPy expressions

```
import sympy as sp

x, y = sp.symbols("x y", real=True)
f = sp.exp(x*y) + x**2 + sp.sin(y)

sp.expand(f)
sp.factor(x**2 - y**2)
sp.simplify(sp.diff(f, x))
```

Assumptions such as `real=True` help SymPy simplify expressions correctly.

Gradient and Hessian in SymPy

```
vars_ = sp.Matrix([x, y])
grad_f = sp.Matrix([sp.diff(f, v) for v in vars_])
H_f = sp.hessian(f, vars_)

print(grad_f)
print(H_f)
```

For a polynomial or rational function, inspect principal minors or eigenvalues. For a non-polynomial function, the domain and parameter assumptions matter.

From symbolic expressions to numerical functions

```
import numpy as np

f_np = sp.lambdify((x, y), f, modules="numpy")
g_np = sp.lambdify((x, y), grad_f, modules="numpy")
H_np = sp.lambdify((x, y), H_f, modules="numpy")

print(f_np(1.0, 2.0))
print(np.asarray(g_np(1.0, 2.0), dtype=float).ravel())
```

`lambdify` connects symbolic derivation to plots, numerical tests, and SciPy.

Task 1: convexity of sets

Decide whether each set is convex. Justify the answer analytically, then write a numerical search for counterexamples.

1. $C_1 = \{x \in \mathbb{R}^2 : \|x\|_2 \leq 1\}.$
2. $C_2 = \{x \in \mathbb{R}^2 : x_1 x_2 \geq 1, x_1 > 0\}.$
3. $C_3 = \{x \in \mathbb{R}^2 : \|x\|_2 \geq 1\}.$
4. $C_4 = \{x \in \mathbb{R}^3 : x_i \geq 0, \mathbf{1}^\top x = 1\}.$

For a nonconvex set, return two feasible points whose midpoint is infeasible.

A numerical convexity probe cannot prove convexity

```
def probe_convexity(sample_point, is_feasible, trials=10_000):  
    for _ in range(trials):  
        x, y = sample_point(), sample_point()  
        t = np.random.random()  
        z = (1 - t)*x + t*y  
        if not is_feasible(z):  
            return False, (x, y, t, z)  
    return True, None
```

Passing random tests supplies evidence, not a proof. A single valid counterexample proves nonconvexity.

Task 2: convexity of functions

Use symbolic Hessians and analytic reasoning.

1. $f_1(x, y) = x^2 + 4xy + 5y^2$ on $\mathbb{R}^2$.
2. $f_2(x, y) = e^{x+y} + x^2$ on $\mathbb{R}^2$.
3. $f_3(x, y) = x^2y^2$ on $\mathbb{R}^2$.
4. $f_4(x, y) = -\log x - \log y$ on $\mathbb{R}_{++}^2$.

State the domain before applying the Hessian test. If the function is not convex, find a point and direction with negative curvature.

Task 3: smoothness and strong convexity

For

$$f(x) = \frac{1}{2}x^\top Qx + b^\top x, \text{quad} Q = \begin{bmatrix} 8 & 2 & 0 \\ 2 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix},$$

complete the following:

1. verify that Q is symmetric positive definite;
2. compute the tight constants μ and L ;
3. calculate $\kappa = L/\mu$;
4. test the smoothness and strong-convexity inequalities on random pairs;
5. predict how scaling Q by 10 changes μ , L , and κ .

Task 4: equality-constrained optimization

Solve analytically and with SymPy:

$$\min_{x,y} (x-1)^2 + 2(y+1)^2 \quad \text{subject to} \quad x + 2y = 1.$$

Required steps:

1. construct $\mathcal{L}(x, y, \nu)$;
2. solve $\nabla_{x,y} \mathcal{L} = 0$ and the constraint;
3. verify second-order sufficiency on the tangent direction;
4. compare the result with direct substitution.

SymPy template for Lagrange multipliers

```
x, y, nu = sp.symbols("x y nu", real=True)
f = (x - 1)**2 + 2*(y + 1)**2
h = x + 2*y - 1
Lagrangian = f + nu*h

equations = [sp.diff(Lagrangian, v) for v in (x, y)] + [h]
solution = sp.solve(equations, (x, y, nu), dict=True)
```

Always substitute the candidate back into the original constraint and objective.

Task 5: KKT with an inequality

Solve

$$\min_{x,y} (x-2)^2 + (y-1)^2 \quad \text{subject to} \quad x+y \leq 1, \quad x \geq 0, \quad y \geq 0.$$

Write every inequality in the form $g_i(x, y) \leq 0$.

1. list possible active sets;
2. solve KKT for each plausible active set;
3. reject candidates violating primal or dual feasibility;
4. verify complementary slackness;
5. explain the geometry of the solution.

Active-set table

For each candidate, fill the table.

Candidate	Active constraints	Primal feasible	$\lambda \geq 0$	KKT
A				
B				
C				

An active constraint may have a zero multiplier. An inactive constraint must have a zero multiplier.

SciPy verification

```
from scipy.optimize import minimize

def objective(z):
    x, y = z
    return (x - 2)**2 + (y - 1)**2

constraints = [{
    "type": "ineq",
    "fun": lambda z: 1.0 - z[0] - z[1]
}]

result = minimize(objective, x0=[0.3, 0.3],
                  bounds=[(0, None), (0, None)],
                  constraints=constraints, method="SLSQP")
```

SciPy uses $c(x) \geq 0$ for inequality dictionaries, while our KKT convention uses $g(x) \leq 0$.

Task 6: simplex-constrained regression

Let

$$\min_w \frac{1}{2} \|Xw - y\|^2 \quad \text{subject to} \quad w_i \geq 0, quad \mathbf{1}^\top w = 1.$$

1. generate a reproducible $X \in \mathbb{R}^{20 \times 4}$ and $y \in \mathbb{R}^{20}$;
2. solve with SLSQP;
3. identify active nonnegativity constraints;
4. estimate multipliers from stationarity;
5. check complementary slackness numerically.

Use explicit tolerances. Do not classify a value as zero using exact equality.

Task 7: sensitivity of a resource constraint

For $R > 0$, define

$$p(R) = \min_{x,y} (x-2)^2 + (y-1)^2 \quad \text{subject to} \quad x+y \leq R.$$

At $R = 1$:

1. solve KKT and obtain the multiplier λ^* ;
2. solve numerically at $R - \varepsilon$, R , and $R + \varepsilon$;
3. estimate $p'(R)$ with a central difference;
4. compare the estimate with $-\lambda^*$;
5. explain when the sensitivity formula may fail to be differentiable.

Task 8: SVM KKT interpretation

Train a linear soft-margin SVM on a small two-dimensional dataset using a standard library, then inspect:

- ▶ points strictly outside the margin;
- ▶ points on the margin;
- ▶ points inside the margin or misclassified;
- ▶ the support-vector coefficients.

Connect these groups to complementary slackness. Explain why only support vectors contribute to the final classifier.

Numerical KKT residuals

For a computed candidate, report separate residuals:

$$r_{\text{stat}} = \|\nabla_x \mathcal{L}\| ,$$

$$r_{\text{eq}} = \|h(x)\| ,$$

$$r_{\text{ineq}} = \|\max(g(x), 0)\| ,$$

$$r_{\text{dual}} = \|\min(\lambda, 0)\| ,$$

$$r_{\text{comp}} = \|\lambda \odot g(x)\| .$$

A single solver success flag cannot replace these checks.

Common implementation errors

- ▶ Mixing the signs $g(x) \leq 0$ and $c(x) \geq 0$.
- ▶ Forgetting that equality multipliers may have either sign.
- ▶ Solving stationarity without checking feasibility.
- ▶ Treating every active constraint as having a positive multiplier.
- ▶ Applying a Hessian convexity test outside the stated domain.
- ▶ Reporting sampled evidence as a proof of convexity.
- ▶ Reading solver multipliers without checking the package convention.